DBMS

Complete Crash Course

Databases · SQL · Normalization · Transactions · Interviews

#	Topic
Part 1	Introduction to DBMS
Part 2	DBMS Architecture
Part 3	Data Models
Part 4	ER Model
Part 5	Relational Model
Part 6	Keys in DBMS
Part 7	SQL — Complete Reference
Part 8	Constraints
Part 9	Normalization (1NF to 4NF)
Part 10	Transactions & ACID Properties
Part 11	Concurrency Control
Part 12	Indexing
Part 13	File Organization
Part 14	Relational Algebra
Part 15	NoSQL Databases & CAP Theorem
Part 16	Database Recovery
Part 17	Advanced SQL

#	Topic
Part 18	Database Design Example
Part 19	20 Interview Questions

PART 1 — INTRODUCTION TO DBMS

1.1 What is Data?

Data is a collection of raw, unorganized facts and figures. On its own, data has no meaning.

```
9876543210, Rahul, Delhi, 25
-- This is data. Without context, it has no meaning.
```

1.2 What is Information?

When data is processed, organized, and given context, it becomes information.

```
Name: Rahul   |   Age: 25   |   City: Delhi   |   Phone: 9876543210
```

1.3 What is a Database?

A database is an organized collection of structured data stored electronically so that it can be easily accessed, managed, and updated. Think of it as a digital filing cabinet where every drawer is labeled and every file is indexed.

Real-world examples:

- Gmail stores your emails, contacts, and settings in a database
- Amazon stores every product, order, and customer in a database
- Your college stores student records, marks, and attendance in a database

1.4 What is a DBMS?

A **Database Management System (DBMS)** is a software system that acts as an interface between the user and the database. It allows users to create, read, update, and delete data while managing security, concurrency, and integrity behind the scenes.

Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server, MongoDB, SQLite

1.5 File System vs DBMS

Problem in File System	How DBMS Solves It
Data Redundancy — same data stored in multiple files	Normalization removes duplicate data
Data Inconsistency — one file updated, other not	Single source of truth; constraints enforce consistency
Difficult Data Access — need custom programs	SQL allows flexible queries without programming
No Security — any program can access any file	Access control, user roles and permissions
No Atomicity — partial writes cause corruption	Transactions ensure all-or-nothing operations
No Concurrent Access — two users cause errors	Concurrency control with locking and timestamps

1.6 Advantages of DBMS

- Reduced Data Redundancy — Data is stored once and referenced everywhere
- Data Integrity — Rules (constraints) prevent invalid data

• Data Security — Role-based access for sensitive data
• Data Abstraction — Users interact without worrying about physical storage
• Concurrent Access — Multiple users can read/write simultaneously
• Backup and Recovery — Automatic mechanisms restore data after crashes
• Data Independence — Changes to storage don't affect application code

PART 2 — DBMS ARCHITECTURE

2.1 Three-Schema Architecture (ANSI/SPARC)

The three-schema architecture separates the database into three levels so that changes at one level do not affect the other levels. This is the foundation of data independence.

```
+----------------------------+
|      External Level        |  <-- What individual users see (Views)
+----------------------------+
             |
+----------------------------+
|      Conceptual Level      |  <-- Logical structure of the entire database
+----------------------------+
             |
+----------------------------+
|      Internal Level        |  <-- Physical storage on disk
+----------------------------+
```

External Level (View Level): Each user or application sees only a portion of the database relevant to them. Example: A bank teller sees account balance and transactions — not internal credit scores.

Conceptual Level (Logical Level): Describes what data is stored and the relationships between data. Defined using tables, columns, data types, and constraints.

Internal Level (Physical Level): Describes how data is physically stored on disk — file organization, indexes, compression, and storage allocation.

2.2 Data Independence

Logical Data Independence: Ability to change the conceptual schema without affecting user views. Example: Adding a new column to a table — existing applications still work.

Physical Data Independence: Ability to change physical storage without affecting the conceptual schema. Example: Moving data from HDD to SSD — queries are unchanged.

2.3 DBMS Architecture — 1-Tier, 2-Tier, 3-Tier

Architecture	Description	Example
1-Tier	Database, application, and UI on the same machine	SQLite in a local desktop app
2-Tier	Client communicates directly with database server	Java app connecting to MySQL server
3-Tier (Industry Standard)	Presentation + Application + Database layers	Browser → Node.js → PostgreSQL

```
[ Browser / Mobile App ]
          |
[ Application Server (Node.js / Spring / Django) ]
          |
[ Database Server (MySQL / PostgreSQL / Oracle) ]
```

PART 3 — DATA MODELS

3.1 What is a Data Model?

A data model defines how data is structured, stored, and accessed in a database. It is the blueprint of the database.

3.2 Types of Data Models

Model	Description	Example / Notes
Hierarchical	Tree structure — each child has one parent	File system directories; cannot represent M:N
Network	Extension of hierarchical; child can have multiple parents	Early airline reservation systems
Relational (Most Used)	Data in tables; relationships via keys (not pointers)	MySQL, PostgreSQL, Oracle — proposed by E.F. Codd 1970
Object-Oriented	Data stored as objects; supports inheritance	ObjectDB
Document (NoSQL)	Data stored as JSON/BSON documents; schema-less	MongoDB

PART 4 — ENTITY-RELATIONSHIP (ER) MODEL

4.1 What is the ER Model?

The ER Model is a conceptual data model that describes the structure of a database using entities, attributes, and relationships. Introduced by Peter Chen in 1976. It is the first step in database design — before writing a single line of SQL.

4.2 Entity

An entity is a real-world object about which data is stored.

Type	Description	Example
Strong Entity	Exists independently; has its own primary key; rectangle symbol	Student, Employee, Product
Weak Entity	Cannot exist without a strong entity; uses partial key; double rectangle	Dependent (of Employee), Order Item (of Order)

4.3 Attribute Types

Type	Description	Example
Simple	Cannot be divided further	age, salary
Composite	Can be divided into sub-attributes	full_name → first_name + last_name
Single-valued	Holds one value	date_of_birth
Multi-valued	Holds multiple values	phone_numbers
Derived	Calculated from another attribute	age derived from date_of_birth
Key	Uniquely identifies an entity	student_id, email

4.4 Cardinality (Mapping Constraints)

Type	Description	Example
One-to-One (1:1)	One instance associated with exactly one other	Person → Passport
One-to-Many (1:N)	One instance associated with multiple others	Department → Employees
Many-to-Many (M:N)	Multiple instances on both sides	Student ↔ Course (requires junction table)

4.5 Participation Constraints

Total Participation: Every entity MUST participate. Shown with double lines. Example: Every Employee must work in a Department.

Partial Participation: Not every entity needs to participate. Shown with single lines. Example: Not every Employee manages a Department.

4.6 Extended ER — Generalization, Specialization, Aggregation

Concept	Direction	Description	Example
Generalization	Bottom-up	Combine lower-level entities into a higher-level entity	Car, Truck, Bike → Vehicle
Specialization	Top-down	Divide a higher-level entity into sub-entities	Employee → Manager, Engineer, Clerk
Aggregation	Treating a relationship as an entity	Used when a relationship participates in another relationship	Employee-Project aggregation managed by Manager

PART 5 — RELATIONAL MODEL

5.1 Terminology

Relational Model Term	Common Term	Description
Relation	Table	A set of tuples (rows)
Tuple	Row / Record	A single entry in the table
Attribute	Column / Field	A named property of the relation
Domain	Set of allowed values	e.g., domain of 'age' = positive integers
Degree	Number of columns	Count of attributes in the relation
Cardinality	Number of rows	Count of tuples in the relation

Example Table: Student

student_id	name	age	city
101	Rahul	21	Delhi
102	Priya	22	Mumbai
103	Arjun	20	Pune

Degree = 4 (four columns) | Cardinality = 3 (three rows) | Domain of age = positive integers

5.2 Properties of a Relation

- Each relation has a unique name
- Each attribute has a distinct name within the relation
- Values in a column must be from the same domain
- Each tuple is unique — no two rows are identical
- Order of tuples and attributes does not matter
- Each cell contains exactly one atomic (indivisible) value

PART 6 — KEYS IN DBMS

Key Type	Description	Example
Super Key	Any attribute(s) that uniquely identify a tuple (may have redundant attributes)	(emp_id), (emp_id, name), (email)
Candidate Key	Minimal super key — no attribute can be removed	(emp_id), (email) — if emails are unique
Primary Key	Chosen candidate key; NOT NULL, must be unique, should be stable	student_id INT PRIMARY KEY
Alternate Key	Candidate key not chosen as primary key	email when student_id is PK
Foreign Key	References primary key of another table; enforces referential integrity	dept_id in Employee → Department
Composite Key	Primary key consisting of two or more attributes	(student_id, course_id) in Enrollment
Surrogate Key	Artificially generated key (e.g., AUTO_INCREMENT); no business meaning	order_id INT AUTO_INCREMENT PRIMARY KEY

Foreign Key — SQL Example

```
CREATE TABLE Department (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(100)
);

CREATE TABLE Employee (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(100),
    dept_id  INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Composite Key — SQL Example

```
CREATE TABLE Enrollment (
    student_id      INT,
    course_id       INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id)
);
```

PART 7 — SQL (STRUCTURED QUERY LANGUAGE)

7.1 Categories of SQL Commands

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Structure of the database
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Data inside tables
DQL	Data Query Language	SELECT	Querying/reading data
DCL	Data Control Language	GRANT, REVOKE	Access and permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Transaction management

7.2 DDL — CREATE, ALTER, DROP, TRUNCATE

```
-- Create database and table
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(100) UNIQUE,
    age        INT CHECK (age >= 18),
    dept_id    INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ALTER: add, modify, drop columns
ALTER TABLE Student ADD phone VARCHAR(15);
ALTER TABLE Student MODIFY age SMALLINT;
ALTER TABLE Student DROP COLUMN phone;

-- DROP vs TRUNCATE
DROP TABLE Student;      -- removes structure AND data permanently
TRUNCATE TABLE Student;  -- removes all rows, keeps structure, resets AUTO_INCREMENT
```

DROP vs TRUNCATE vs DELETE

Feature	DROP	TRUNCATE	DELETE
Removes structure	Yes	No	No
Removes data	Yes (all)	Yes (all)	Yes (specific rows)
WHERE clause	No	No	Yes
Can rollback	No	No (generally)	Yes

Feature	DROP	TRUNCATE	DELETE
Resets AUTO_INCREMENT	N/A	Yes	No
Speed	Fast	Fast	Slow (row-by-row)

7.3 DML — INSERT, UPDATE, DELETE

```
-- INSERT
INSERT INTO Student (student_id, name, email, age, dept_id)
VALUES (101, 'Rahul Sharma', 'rahul@example.com', 21, 10);

-- Multiple rows
INSERT INTO Student VALUES
    (102, 'Priya Patel', 'priya@example.com', 22, 10),
    (103, 'Arjun Singh', 'arjun@example.com', 20, 20);

-- UPDATE
UPDATE Student SET email = 'new@example.com' WHERE student_id = 101;
UPDATE Student SET age = 22, dept_id = 20 WHERE student_id = 101;

-- DELETE
DELETE FROM Student WHERE student_id = 104;
DELETE FROM Student WHERE age < 18;
```

7.4 SELECT — WHERE, ORDER BY, LIMIT

```
SELECT * FROM Student;
SELECT name, email FROM Student;
SELECT name AS student_name FROM Student;

-- WHERE conditions
SELECT * FROM Student WHERE age > 20 AND dept_id = 10;
SELECT * FROM Student WHERE dept_id IN (10, 20, 30);
SELECT * FROM Student WHERE age BETWEEN 18 AND 22;
SELECT * FROM Student WHERE name LIKE 'R%';    -- starts with R
SELECT * FROM Student WHERE name LIKE '%rah%'; -- contains rah
SELECT * FROM Student WHERE email IS NULL;

-- ORDER BY and LIMIT
SELECT * FROM Student ORDER BY age DESC;
SELECT * FROM Student ORDER BY dept_id ASC, age DESC;
SELECT * FROM Student LIMIT 5 OFFSET 10;  -- pagination: skip 10, get 5
```

7.5 Aggregate Functions

```
SELECT COUNT(*) FROM Student;
SELECT COUNT(email) FROM Student;  -- counts only non-NULL
SELECT SUM(age), AVG(age), MAX(age), MIN(age) FROM Student;
```

7.6 GROUP BY and HAVING

GROUP BY groups rows with the same value; HAVING filters those groups.

```
-- Students per department
SELECT dept_id, COUNT(*) AS student_count
FROM Student
GROUP BY dept_id;

-- Only departments with more than 5 students
SELECT dept_id, COUNT(*) AS student_count
FROM Student
GROUP BY dept_id
HAVING COUNT(*) > 5;

-- WHERE + GROUP BY + HAVING combined
SELECT dept_id, COUNT(*) AS student_count
FROM Student
WHERE age >= 18
GROUP BY dept_id
HAVING COUNT(*) > 3;
```

Clause	When it filters	Can use aggregates?
WHERE	Before grouping — filters individual rows	No
HAVING	After grouping — filters groups	Yes

7.7 SQL Execution Order

```
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
```

This is why you CANNOT use a SELECT alias in a WHERE clause — WHERE executes before SELECT.

7.8 JOINS

Joins combine rows from two or more tables based on a related column.

```
-- Setup
CREATE TABLE Department (dept_id INT PRIMARY KEY, dept_name VARCHAR(100));
INSERT INTO Department VALUES (10,'Computer Science'),(20,'Electronics'),(30,'Mechanical');

CREATE TABLE Employee (emp_id INT PRIMARY KEY, emp_name VARCHAR(100), dept_id INT);
INSERT INTO Employee VALUES (1,'Rahul',10),(2,'Priya',20),(3,'Arjun',10),(4,'Sneha',99);
-- Note: Sneha has dept_id=99 which does NOT exist in Department
```

Join Type	Returns	NULL for
INNER JOIN	Rows with matching values in BOTH tables	Sneha is excluded (no match)
LEFT JOIN	ALL rows from left table + matching from right	Sneha's dept_name is NULL
RIGHT JOIN	ALL rows from right table + matching from left	Mechanical's emp_name is NULL
FULL OUTER JOIN	ALL rows from BOTH tables	Both unmatched sides are NULL
CROSS JOIN	Cartesian product (every row × every row)	No NULLs — all combinations
SELF JOIN	Table joined with itself	Used for hierarchical data (employee-manager)

```
-- INNER JOIN
SELECT e.emp_name, d.dept_name
FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id;

-- LEFT JOIN
SELECT e.emp_name, d.dept_name
FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id;

-- FULL OUTER JOIN (MySQL workaround)
SELECT e.emp_name, d.dept_name FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id;

-- SELF JOIN (employee and their manager)
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM Employee e JOIN Employee m ON e.manager_id = m.emp_id;
```

7.9 Subqueries

```
-- Simple subquery
SELECT name, age FROM Student
WHERE age > (SELECT AVG(age) FROM Student);

-- IN subquery
SELECT emp_name FROM Employee
WHERE dept_id IN (SELECT dept_id FROM Department WHERE location = 'Delhi');

-- Correlated subquery (runs once per row of outer query)
SELECT emp_name, salary FROM Employee e
WHERE salary > (SELECT AVG(salary) FROM Employee WHERE dept_id = e.dept_id);
```

7.10 Views

A view is a virtual table created from a SELECT query. It does not store data — it is a saved query.

```
CREATE VIEW CS_Students AS
SELECT student_id, name, email FROM Student WHERE dept_id = 10;

SELECT * FROM CS_Students;  -- use like a regular table

DROP VIEW CS_Students;
```

7.11 Stored Procedures and Triggers

```
-- Stored Procedure
DELIMITER $$
CREATE PROCEDURE GetStudentsByDept(IN dept INT)
BEGIN
    SELECT student_id, name, age FROM Student WHERE dept_id = dept;
END $$
DELIMITER ;

CALL GetStudentsByDept(10);

-- Trigger: log every deletion
CREATE TRIGGER before_student_delete
BEFORE DELETE ON Student FOR EACH ROW
BEGIN
    INSERT INTO Student_Audit (student_id) VALUES (OLD.student_id);
END;
```

7.12 DCL and TCL

```
-- DCL
GRANT SELECT, INSERT ON college_db.Student TO 'john'@'localhost';
REVOKE INSERT ON college_db.Student FROM 'john'@'localhost';

-- TCL (Bank transfer example)
START TRANSACTION;
    UPDATE Account SET balance = balance - 5000 WHERE account_id = 'A';
    UPDATE Account SET balance = balance + 5000 WHERE account_id = 'B';
COMMIT;      -- both changes saved permanently
-- OR:
ROLLBACK;    -- undo ALL changes since START TRANSACTION
```

PART 8 — CONSTRAINTS

Constraint	Purpose	Example
PRIMARY KEY	Unique + NOT NULL; one per table	student_id INT PRIMARY KEY
FOREIGN KEY	References PK of another table; referential integrity	FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
UNIQUE	All values in column must be distinct	email VARCHAR(100) UNIQUE
NOT NULL	Column cannot be empty	name VARCHAR(100) NOT NULL
CHECK	Values must meet a condition	age INT CHECK (age >= 18)
DEFAULT	Default value if none provided	status VARCHAR(20) DEFAULT 'active'

```
CREATE TABLE Product (
    product_id   INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price        DECIMAL(10,2) CHECK (price > 0),
    category     VARCHAR(50) DEFAULT 'General',
    sku          VARCHAR(50) UNIQUE,
    supplier_id  INT,
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
);
```

PART 9 — NORMALIZATION

9.1 What is Normalization?

Normalization is the process of organizing a relational database to reduce data redundancy and prevent data anomalies (insert, update, delete). Introduced by Edgar F. Codd alongside the relational model.

9.2 Anomalies — Why We Normalize

Anomaly	Description
Insert Anomaly	Cannot add a new product to the catalog without creating a fake order
Update Anomaly	If a phone number changes, it must be updated in every row — miss one and data is inconsistent
Delete Anomaly	Deleting an order removes all information about the product in that order

9.3 First Normal Form (1NF)

Rule: Every column must contain atomic (indivisible) values. No repeating groups or multi-valued attributes.

```
-- VIOLATION (courses is not atomic)
student_id | name  | courses
101        | Rahul | DBMS, OS, CN

-- AFTER 1NF
student_id | name  | course
101        | Rahul | DBMS
101        | Rahul | OS
101        | Rahul | CN
```

9.4 Second Normal Form (2NF)

Prerequisite: Must be in 1NF. **Rule:** No partial dependency — every non-key attribute must depend on the ENTIRE composite primary key.

```
-- VIOLATION: Enrollment(student_id, course_id, student_name, course_name, grade)
-- PK = (student_id, course_id)
-- student_name depends ONLY on student_id  -> partial dependency
-- course_name  depends ONLY on course_id   -> partial dependency
-- grade depends on BOTH student_id+course_id -> OK

-- AFTER 2NF (split into 3 tables)
Student(student_id, student_name)
Course(course_id, course_name)
Enrollment(student_id, course_id, grade)
```

9.5 Third Normal Form (3NF)

Prerequisite: Must be in 2NF. **Rule:** No transitive dependency — a non-key attribute should not depend on another non-key attribute.

```
-- VIOLATION: Employee(emp_id, emp_name, dept_id, dept_name)
-- emp_id -> dept_id  (OK)
-- dept_id -> dept_name  (transitive -- dept_name depends on dept_id, not emp_id)

-- AFTER 3NF
Employee(emp_id, emp_name, dept_id)
Department(dept_id, dept_name)
```

9.6 Boyce-Codd Normal Form (BCNF)

Rule: For every non-trivial functional dependency X → Y, X must be a super key. Stricter than 3NF.

9.7 Normalization Summary

Normal Form	Prerequisite	Eliminates
1NF	None	Multi-valued / composite attributes — ensures atomicity
2NF	1NF	Partial dependencies (non-key attribute depends on part of composite PK)
3NF	2NF	Transitive dependencies (non-key depends on non-key)
BCNF	3NF	Anomalies where non-superkeys determine attributes
4NF	BCNF	Multi-valued dependencies

9.8 Functional Dependency

A functional dependency X → Y means that for any two tuples with the same value of X, they must have the same value of Y.

FD Type	Description	Example
Trivial	Y is a subset of X	{student_id, name} → student_id
Non-trivial	Y is NOT a subset of X	student_id → name
Partial Dependency	Non-key attribute depends on part of composite PK	student_name depends only on student_id in Enrollment
Transitive Dependency	A→B and B→C, so A→C transitively	emp_id → dept_id → dept_name

PART 10 — TRANSACTIONS & ACID PROPERTIES

10.1 What is a Transaction?

A transaction is a logical unit of work that consists of one or more operations that must ALL succeed or ALL fail together.

```
-- Bank Transfer: Rs. 5000 from Account A to Account B
START TRANSACTION;
    UPDATE Account SET balance = balance - 5000 WHERE account_id = 'A';
    UPDATE Account SET balance = balance + 5000 WHERE account_id = 'B';
COMMIT;
-- If system crashes between the two UPDATEs without a transaction,
-- Rs.5000 is deducted from A but never credited to B. Money disappears.
```

10.2 ACID Properties

Property	Definition	How it's Enforced
Atomicity (A)	All operations succeed OR all are rolled back — no partial completion	Transaction Manager / ROLLBACK on failure
Consistency (C)	Transaction brings DB from one valid state to another; all constraints satisfied	Integrity constraints, triggers, application logic
Isolation (I)	Concurrent transactions behave as if they are the only transaction running	Locking, MVCC, isolation levels
Durability (D)	Committed changes survive system failures — permanent on disk	Write-Ahead Logging (WAL), checkpoints

10.3 Isolation Levels

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
Read Uncommitted	Possible	Possible	Possible	Highest
Read Committed	Prevented	Possible	Possible	High
Repeatable Read	Prevented	Prevented	Possible	Medium
Serializable	Prevented	Prevented	Prevented	Lowest

10.4 Concurrency Problems

Problem	Description
Dirty Read	T2 reads data modified but NOT YET COMMITTED by T1; T1 then rolls back — T2 used phantom data
Non-Repeatable Read	T1 reads a row; T2 updates and commits it; T1 reads same row and gets a different value
Phantom Read	T1 queries rows matching a condition; T2 inserts a new matching row; T1 re-runs query and gets an extra row
Lost Update	T1 and T2 both read a value, both update it — T1's update is silently overwritten by T2

10.5 Transaction States

```
BEGIN
  |
  v
ACTIVE  (executing)
  |
  +-- success --> PARTIALLY COMMITTED --> COMMITTED (permanent)
  |
  +-- failure --> FAILED --> ABORTED (rolled back)
```

PART 11 — CONCURRENCY CONTROL

11.1 Lock-Based Protocols

Lock Type	Operation	Compatibility
Shared Lock (S / Read Lock)	READ only	Multiple transactions can hold S-lock simultaneously
Exclusive Lock (X / Write Lock)	READ + WRITE	Only ONE transaction can hold X-lock; no other lock allowed simultaneously

11.2 Two-Phase Locking (2PL)

2PL guarantees serializability. It has two phases:

- **Growing Phase:** Transaction acquires locks; cannot release any lock yet
- **Shrinking Phase:** Transaction releases locks; cannot acquire new locks

*The point where the last lock is acquired is called the **lock point**. Problem: 2PL can cause deadlocks.*

11.3 Deadlock

A deadlock occurs when two or more transactions are waiting for each other to release locks, creating a circular dependency.

```
T1 holds lock on A, waiting for B
T2 holds lock on B, waiting for A
--> Neither can proceed. DEADLOCK.
```

Approach	Method
Detection	Build a wait-for graph. If a cycle exists, deadlock exists. Abort one transaction (the victim).
Prevention — Wait-Die	Older transaction waits; younger transaction is aborted (dies) and restarted
Prevention — Wound-Wait	Older transaction preempts (wounds) younger; younger waits

11.4 Timestamp-Based Concurrency Control

Each transaction is assigned a unique timestamp at start. If T tries to read/write an item that has been accessed by a newer transaction, T is rolled back and restarted with a new timestamp.

PART 12 — INDEXING

12.1 What is an Index?

An index is a data structure (usually a B-tree) that improves data retrieval speed at the cost of additional storage and slower writes. Like the index in a textbook — jump directly to the page instead of reading every page.

12.2 Types of Indexes

Index Type	Description	Notes
Primary / Clustered	Built on primary key; data physically sorted by this key	Only ONE per table
Secondary / Non-Clustered	Separate structure; contains indexed values + pointers to rows	Multiple per table allowed
Composite Index	Index on multiple columns	Useful for multi-column WHERE/ORDER BY
Dense Index	One index entry per record	Faster search; more storage; required for non-ordered attributes
Sparse Index	One index entry per data block	Less storage; data must be physically sorted
Hash Index	Uses hash function; O(1) exact lookup	Does NOT support range queries

```
-- Create index
CREATE INDEX idx_student_name ON Student(name);

-- Composite index
CREATE INDEX idx_dept_age ON Student(dept_id, age);

-- Drop index
DROP INDEX idx_student_name ON Student;
```

12.3 B-Tree Index

Most relational databases use B-trees (Balanced Trees). Properties: self-balancing, all leaf nodes at same level, supports range queries (BETWEEN, <, >), O(log n) for search/insert/delete.

12.4 When to Use and Avoid Indexes

Use Index When	Avoid Index When
Column frequently in WHERE, JOIN ON, ORDER BY, GROUP BY	Table is very small (full scan is faster)
Column has high cardinality (many distinct values)	Column has very low cardinality (e.g., gender M/F)
Table is large and reads are frequent	Table has very frequent INSERT/UPDATE/DELETE (slows writes)

PART 13 — FILE ORGANIZATION

Method	Description	Best For
Heap File	Records in no particular order; inserted at end; full scan for retrieval	Small tables, bulk loading
Sequential File	Records sorted by a key field; efficient range queries; expensive insertion	Reporting, sequential processing
Hashing	Hash function on key field; O(1) exact match; poor for range queries	Exact lookups
Clustered	Related records from multiple tables on the same disk block	Frequently joined tables

PART 14 — RELATIONAL ALGEBRA

Relational Algebra is a procedural query language that operates on relations and returns a relation. It is the theoretical foundation of SQL.

Operation	Symbol	SQL Equivalent	Example
Selection	σ (sigma)	WHERE	σ(age>20)(Student) — rows where age > 20
Projection	π (pi)	SELECT columns	π(name,email)(Student) — only name, email
Union	∪	UNION	Students_Delhi ∪ Students_Mumbai
Intersection	∩	INTERSECT	Rows present in both relations
Difference	–	EXCEPT / MINUS	Rows in R1 but not R2
Cartesian Product	×	CROSS JOIN	All combinations of rows from two relations
Natural Join	■	INNER JOIN on common attributes	Rows with matching values on shared columns
Rename	ρ (rho)	AS alias	Rename relation or attributes

PART 15 — NoSQL DATABASES & CAP THEOREM

15.1 Types of NoSQL Databases

Type	Data Model	Best For	Examples
Document Store	JSON/BSON documents; schema-less	Content management, user profiles, catalogs	MongoDB
Key-Value Store	Key → Value pairs	Caching, sessions, shopping carts	Redis, DynamoDB
Column-Family	Columns grouped into families	Time-series, analytics, IoT data	Cassandra, HBase
Graph Database	Nodes (entities) + Edges (relationships)	Social networks, fraud detection, recommendations	Neo4j, Amazon Neptune

```
// MongoDB Document Example
{
  "_id": "user_101",
  "name": "Rahul Sharma",
  "age": 25,
  "address": { "city": "Delhi", "pin": "110001" },
  "skills": ["Python", "SQL", "MongoDB"]
}
```

15.2 CAP Theorem

A distributed database can guarantee at most TWO of the three:

Property	Meaning
C — Consistency	Every read receives the most recent write or an error
A — Availability	Every request gets a response (may not be the most recent data)
P — Partition Tolerance	System continues operating despite network partition between nodes

Since network partitions are unavoidable, the real choice is CP vs AP:

Choice	Databases	Use When
CP (Consistency + Partition)	MongoDB, HBase, Zookeeper	Banking, financial systems — cannot tolerate stale data
AP (Availability + Partition)	Cassandra, CouchDB, DynamoDB	Social feeds, DNS — stale data briefly acceptable

15.3 RDBMS vs NoSQL

Feature	RDBMS	NoSQL
Schema	Fixed, predefined	Flexible, dynamic
Query Language	SQL (standardized)	Varies by database

Feature	RDBMS	NoSQL
Scalability	Vertical (scale up)	Horizontal (scale out)
ACID	Full ACID	Eventual consistency (usually)
Best For	Structured data, complex queries	Unstructured data, high volume, speed
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra

PART 16 — DATABASE RECOVERY

16.1 Types of Failures

Failure Type	Cause	Data Lost?
Transaction Failure	Logical error, deadlock, constraint violation	No (rolled back)
System Failure	Power loss, OS crash — volatile memory (RAM) lost; disk data survives	In-memory data only
Media Failure	Disk crash	Permanent — requires backup

16.2 Write-Ahead Logging (WAL)

Before any change is written to the database, it is first written to a log file. This ensures that even if the system crashes mid-transaction, the log can be replayed (redo) or undone (undo) during recovery.

```
Log record format: [Transaction_ID, Operation, Object, Old_Value, New_Value]
```

16.3 Recovery Techniques

Technique	Description
Undo Logging	On failure, all changes by uncommitted transactions are reversed to old values
Redo Logging	Committed transactions whose changes are not yet on disk are written from the log
Undo/Redo	Combines both — uncommitted changes are undone, committed changes not on disk are redone
Checkpointing	Periodically flush all committed transactions to disk and record a checkpoint; recovery only processes logs after last checkpoint

PART 17 — ADVANCED SQL

17.1 Window Functions

Window functions perform calculations across a set of rows related to the current row — without collapsing them into a single group like GROUP BY.

```
-- RANK employees by salary within each department
SELECT emp_name, dept_id, salary,
    RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept,
    DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rank,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS global_row
FROM Employee;

-- Running total
SELECT order_date, amount,
    SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM Sales;

-- LAG / LEAD: access previous/next row value
SELECT emp_name, salary,
    LAG(salary,1)  OVER (ORDER BY salary) AS prev_salary,
    LEAD(salary,1) OVER (ORDER BY salary) AS next_salary
FROM Employee;
```

Function	Behaviour with Ties
ROW_NUMBER()	Unique sequential number — no ties (1, 2, 3, 4)
RANK()	Same rank for ties; gaps after ties (1, 1, 3, 4)
DENSE_RANK()	Same rank for ties; NO gaps (1, 1, 2, 3)
NTILE(n)	Divides rows into n equal buckets

17.2 CTEs and Recursive CTEs

```
-- Simple CTE
WITH DeptStats AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM Employee GROUP BY dept_id
)
SELECT e.emp_name, e.salary, d.avg_salary
FROM Employee e JOIN DeptStats d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_salary;

-- Recursive CTE: org chart
WITH RECURSIVE OrgChart AS (
    SELECT emp_id, emp_name, manager_id, 0 AS level
    FROM Employee WHERE manager_id IS NULL  -- CEO
    UNION ALL
    SELECT e.emp_id, e.emp_name, e.manager_id, o.level + 1
    FROM Employee e JOIN OrgChart o ON e.manager_id = o.emp_id
)
SELECT * FROM OrgChart;
```

17.3 Useful SQL Patterns

```
-- CASE expression
SELECT emp_name, salary,
    CASE WHEN salary > 100000 THEN 'Senior'
         WHEN salary > 50000  THEN 'Mid-Level'
         ELSE 'Junior' END AS grade
FROM Employee;

-- UNION (removes duplicates) vs UNION ALL (keeps all)
SELECT name FROM TableA UNION     SELECT name FROM TableB;
SELECT name FROM TableA UNION ALL SELECT name FROM TableB;

-- EXISTS vs IN (EXISTS more efficient for large subqueries)
SELECT emp_name FROM Employee e
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.emp_id = e.emp_id);
```

PART 18 — COMPLETE DATABASE DESIGN EXAMPLE

E-Commerce Database — full schema applying normalization, constraints, and foreign keys.

```
CREATE TABLE Customer (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name   VARCHAR(150) NOT NULL,
    email       VARCHAR(100) UNIQUE NOT NULL,
    phone       VARCHAR(15),
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Category (
    category_id   INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL
);

CREATE TABLE Product (
    product_id     INT AUTO_INCREMENT PRIMARY KEY,
    product_name   VARCHAR(200) NOT NULL,
    description    TEXT,
    price          DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock_quantity INT DEFAULT 0 CHECK (stock_quantity >= 0),
    category_id    INT,
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
);

CREATE TABLE Orders (
    order_id    INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status      VARCHAR(50) DEFAULT 'Pending',
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

CREATE TABLE OrderItem (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id      INT NOT NULL,
    product_id    INT NOT NULL,
    quantity      INT NOT NULL CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id)   REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);

CREATE TABLE Address (
    address_id  INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    street      VARCHAR(200),
    city        VARCHAR(100),
    state       VARCHAR(100),
    pin_code    VARCHAR(10),
    is_default  BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
```

PART 19 — 20 MOST ASKED DBMS INTERVIEW QUESTIONS

Q1. What is the difference between a database and a DBMS?

A database is an organized collection of data. A DBMS is the software that creates, manages, and provides access to that database. The database is the data itself; the DBMS is the tool. Example: MySQL is the DBMS; the tables and records it manages form the database.

Q2. What are the ACID properties? Explain each with an example.

Atomicity: A bank transfer either fully completes (debit + credit) or is fully rolled back — no partial update.

Consistency: Before and after the transfer, total money in the system remains the same; all constraints satisfied.

Isolation: Two simultaneous transfers behave as if each is the only one running.

Durability: Once the bank confirms the transfer, the record survives even a system crash.

Q3. What is the difference between DELETE, TRUNCATE, and DROP?

DELETE removes specific rows (uses WHERE), is a DML command, logs every row, can be rolled back, does not reset AUTO_INCREMENT.

TRUNCATE removes ALL rows without logging individually, is faster, cannot be rolled back, resets AUTO_INCREMENT, keeps table structure.

DROP removes the entire table (structure + data) permanently. Cannot be rolled back.

Q4. What are the different types of joins? Explain.

INNER JOIN: rows matching in BOTH tables. LEFT JOIN: all left rows + matching right (NULL if no match). RIGHT JOIN: all right rows + matching left (NULL if no match). FULL OUTER JOIN: all rows from both tables (NULLs where no match). CROSS JOIN: Cartesian product (every row × every row). SELF JOIN: table joined with itself — used for employee-manager hierarchy.

Q5. What is normalization? Explain 1NF, 2NF, and 3NF.

Normalization reduces data redundancy and prevents anomalies.

1NF: Every column must have atomic values — no repeating groups or arrays.

2NF: Must be in 1NF + no partial dependencies (every non-key attribute depends on the ENTIRE composite primary key).

3NF: Must be in 2NF + no transitive dependencies (non-key attribute must not depend on another non-key attribute).

Q6. What is the difference between a primary key and a unique key?

Primary Key: must be unique, NOT NULL, only ONE per table, creates a clustered index.

Unique Key: must be unique, can have ONE NULL (in most RDBMS), MULTIPLE unique keys per table, creates a non-clustered index.

Q7. What is a foreign key and what is referential integrity?

A foreign key is an attribute that references the primary key of another table. Referential integrity is the constraint that every foreign key value must either match a primary key in the referenced table OR be NULL. It prevents orphan records — child records with no corresponding parent.

Q8. What is the difference between WHERE and HAVING?

WHERE filters individual rows BEFORE any grouping — cannot use aggregate functions.

HAVING filters groups AFTER GROUP BY is applied — can use aggregate functions.

Execution order: WHERE → GROUP BY → HAVING.

Q9. What is a deadlock? How is it detected and resolved?

A deadlock occurs when two or more transactions each wait for a lock held by the other — circular wait. Detection: build a wait-for graph; if a cycle exists, deadlock exists. Resolution: abort one transaction (the victim — least work done). Prevention: Wait-Die (older waits, younger dies) or Wound-Wait (older preempts younger).

Q10. What is an index? What are its advantages and disadvantages?

An index is a data structure (B-tree or hash) that speeds up data retrieval by creating a quick lookup path.

Advantages: Dramatically faster SELECT, JOIN, ORDER BY on large tables.

Disadvantages: Extra storage space; slows INSERT, UPDATE, DELETE because indexes must also be updated; too many indexes degrade write performance.

Q11. What is the difference between clustered and non-clustered index?

Clustered index: determines the physical order of data in the table — only ONE per table. Usually created by PRIMARY KEY.

Non-clustered index: a separate structure with indexed values + pointers to data rows — multiple per table allowed. Does not affect physical order.

Q12. What is a view? What are its advantages?

A view is a virtual table created from a stored SELECT query. It stores no data — it runs the underlying query each time.

Advantages: Security (expose only specific columns), simplicity (hide complex joins), consistency (multiple apps use the same view), logical independence.

Q13. What are the isolation levels in SQL?

Read Uncommitted: can read uncommitted data (dirty reads possible).

Read Committed: reads only committed data (prevents dirty reads).

Repeatable Read: prevents non-repeatable reads — data read cannot be changed by others until transaction ends.

Serializable: strictest — transactions are fully isolated (prevents phantom reads). Higher isolation = fewer problems but lower throughput.

Q14. What is the difference between OLTP and OLAP?

OLTP (Online Transaction Processing): day-to-day transactions, short INSERT/UPDATE/DELETE queries, normalized (3NF), current operational data. Examples: banking app, e-commerce checkout.

OLAP (Online Analytical Processing): data analysis, complex long-running SELECT with aggregations, denormalized (star/snowflake schema), historical large-volume data. Examples: BI dashboards, data warehouses.

Q15. What is the difference between SQL and NoSQL?

SQL: relational, fixed schema, ACID transactions, vertical scaling, best for structured data with complex relationships (banking, ERP).

NoSQL: non-relational, flexible schema, eventual consistency, horizontal scaling, best for unstructured high-volume data (social feeds, real-time analytics, IoT).

Q16. What is a stored procedure and how is it different from a function?

Stored Procedure: precompiled SQL statements called explicitly (CALL); can perform INSERT/UPDATE/DELETE; optional return value; can manage transactions.

Function: must return a single value; used within SQL expressions; generally read-only (no DML); cannot manage transactions. Procedures are for business logic; functions are for computations.

Q17. Explain transaction states.

Active: transaction is executing.

Partially Committed: all operations done, awaiting COMMIT.

Committed: changes permanently saved to disk.

Failed: error occurred — must be rolled back.

Aborted: rolled back to previous consistent state.

Q18. What is BCNF and when does 3NF differ from BCNF?

BCNF: for every non-trivial FD $X \rightarrow Y$, X must be a super key — stricter than 3NF.

3NF allows a non-super key on the left side if the right side is part of some candidate key. They differ when there are overlapping candidate keys — a relation can be in 3NF but not BCNF.

Q19. What is the CAP theorem and how does it guide database selection?

A distributed system can guarantee at most 2 of: Consistency, Availability, Partition Tolerance. Since partitions are unavoidable, the choice is CP vs AP.

CP (MongoDB, HBase): consistent but may be unavailable during partition — use for banking.

AP (Cassandra, CouchDB): always available but may return stale data — use for social feeds where brief inconsistency is acceptable.

Q20. Write a query to find the second highest salary.

```
-- Method 1: LIMIT + OFFSET
SELECT DISTINCT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Method 2: Subquery
SELECT MAX(salary) FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Method 3: DENSE_RANK (most robust — handles ties correctly)
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM Employee
)
SELECT salary FROM RankedSalaries WHERE rnk = 2;

-- Generalized: Nth highest salary (replace 2 with N)
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM Employee
)
SELECT salary FROM RankedSalaries WHERE rnk = N;
```

End of DBMS Crash Course Notes

Topics Covered: Introduction · Architecture · Data Models · ER Model · Relational Model · Keys · SQL (DDL/DML/DQL/DCL/TCL) · Joins · Subqueries · Views · Stored Procedures · Triggers · Constraints · Normalization (1NF–4NF) · Functional Dependencies · ACID & Transactions · Concurrency Control · Indexing · File Organization · Relational Algebra · NoSQL & CAP Theorem · Database Recovery · Advanced SQL · 20 Interview Questions